

OBSERVATION TASK 5

Title

Collection Framework – Method Implementation – Part 1

Objective / Aim

To study and implement the important methods of the **Java Collection Framework** using different collection interfaces such as **Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and ListIterator**.

The program demonstrates how to **add, remove, search, retrieve, update, sort, iterate, and navigate through elements** using the appropriate collection methods.

Problem Statement

Develop a Java program to demonstrate the important methods provided by the Java Collection Framework.

The program should demonstrate:

- Collection interface methods
- List interface methods
- Set interface methods
- SortedSet interface methods
- NavigableSet interface methods
- Queue interface methods
- Deque interface methods
- Map interface methods
- SortedMap interface methods
- NavigableMap interface methods
- Iterator methods
- ListIterator methods

The program should display the result of each operation clearly.

Collection Framework Interfaces and Methods

1. Collection Interface

The `Collection` interface provides basic operations common to most collections.

Method	Description
<code>add(E e)</code>	Adds an element to the collection
<code>addAll(Collection c)</code>	Adds all elements of another collection
<code>remove(Object o)</code>	Removes the specified element
<code>removeAll(Collection c)</code>	Removes all matching elements
<code>contains(Object o)</code>	Checks whether an element exists
<code>containsAll(Collection c)</code>	Checks whether all specified elements exist
<code>size()</code>	Returns the number of elements
<code>isEmpty()</code>	Checks whether the collection is empty
<code>clear()</code>	Removes all elements
<code>iterator()</code>	Returns an iterator

2. List Interface

The `List` interface maintains insertion order and allows duplicate elements.

Method	Description
<code>add(E e)</code>	Adds an element
<code>add(index, element)</code>	Inserts an element at a specified position

Method	Description
<code>get(index)</code>	Retrieves an element
<code>set(index, element)</code>	Replaces an element
<code>remove(index)</code>	Removes an element at an index
<code>indexOf(Object o)</code>	Returns the first occurrence
<code>lastIndexOf(Object o)</code>	Returns the last occurrence
<code>subList(from, to)</code>	Returns a portion of the list
<code>sort(Comparator)</code>	Sorts the list

3. Set Interface

The `Set` interface does not allow duplicate elements.

Method	Description
<code>add(E e)</code>	Adds an element if it is not already present
<code>remove(Object o)</code>	Removes an element
<code>contains(Object o)</code>	Checks whether an element exists
<code>size()</code>	Returns the number of elements
<code>isEmpty()</code>	Checks whether the set is empty
<code>clear()</code>	Removes all elements

4. SortedSet Interface

`SortedSet` stores elements according to their natural ascending order.

Method	Description
<code>first()</code>	Returns the smallest element
<code>last()</code>	Returns the largest element
<code>headSet()</code>	Returns elements less than the specified element
<code>tailSet()</code>	Returns elements greater than or equal to the specified element
<code>subSet()</code>	Returns elements within a specified range
<code>comparator()</code>	Returns the comparator used for ordering

5. NavigableSet Interface

`NavigableSet` provides additional navigation methods over a sorted set.

Method	Description
<code>lower()</code>	Returns the greatest element less than the specified element
<code>floor()</code>	Returns the greatest element less than or equal to the specified element
<code>ceiling()</code>	Returns the smallest element greater than or equal to the specified element
<code>higher()</code>	Returns the smallest element greater than the specified element
<code>pollFirst()</code>	Removes and returns the first element
<code>pollLast()</code>	Removes and returns the last element
<code>descendingSet()</code>	Returns the set in reverse order

6. Queue Interface

A `Queue` generally follows **FIFO (First In, First Out)** order.

Method	Description
<code>add()</code>	Adds an element
<code>offer()</code>	Adds an element
<code>remove()</code>	Removes and returns the head
<code>poll()</code>	Removes and returns the head
<code>element()</code>	Retrieves the head without removing it
<code>peek()</code>	Retrieves the head without removing it

7. Deque Interface

A Deque is a **double-ended queue** that allows insertion and removal from both ends.

Method	Description
<code>addFirst()</code>	Adds an element at the front
<code>addLast()</code>	Adds an element at the end
<code>offerFirst()</code>	Offers an element at the front
<code>offerLast()</code>	Offers an element at the end
<code>removeFirst()</code>	Removes the first element
<code>removeLast()</code>	Removes the last element
<code>pollFirst()</code>	Retrieves and removes the first element
<code>pollLast()</code>	Retrieves and removes the last element
<code>peekFirst()</code>	Retrieves the first element
<code>peekLast()</code>	Retrieves the last element

8. Map Interface

A `Map` stores data in **key-value pairs**.

Method	Description
<code>put ()</code>	Adds a key-value pair
<code>get ()</code>	Retrieves a value using its key
<code>remove ()</code>	Removes a mapping
<code>containsKey ()</code>	Checks whether a key exists
<code>containsValue ()</code>	Checks whether a value exists
<code>keySet ()</code>	Returns all keys
<code>values ()</code>	Returns all values
<code>entrySet ()</code>	Returns all key-value mappings
<code>size ()</code>	Returns the number of mappings
<code>isEmpty ()</code>	Checks whether the map is empty
<code>clear ()</code>	Removes all mappings

9. SortedMap Interface

`SortedMap` maintains its keys in ascending order.

Method	Description
<code>firstKey ()</code>	Returns the smallest key
<code>lastKey ()</code>	Returns the largest key
<code>headMap ()</code>	Returns mappings with keys less than the specified key
<code>tailMap ()</code>	Returns mappings with keys greater than or equal to the specified key

Method	Description
<code>subMap()</code>	Returns mappings within a specified key range
<code>comparator()</code>	Returns the comparator

10. NavigableMap Interface

`NavigableMap` provides navigation operations on sorted key-value mappings.

Method	Description
<code>lowerKey()</code>	Returns the greatest key less than the specified key
<code>floorKey()</code>	Returns the greatest key less than or equal to the specified key
<code>ceilingKey()</code>	Returns the smallest key greater than or equal to the specified key
<code>higherKey()</code>	Returns the smallest key greater than the specified key
<code>firstEntry()</code>	Returns the first key-value mapping
<code>lastEntry()</code>	Returns the last key-value mapping
<code>pollFirstEntry()</code>	Removes and returns the first mapping
<code>pollLastEntry()</code>	Removes and returns the last mapping
<code>descendingMap()</code>	Returns the map in reverse order

11. Iterator Interface

`Iterator` is used to traverse elements of a collection.

Method	Description
<code>hasNext()</code>	Checks whether another element exists
<code>next()</code>	Returns the next element

Method	Description
<code>remove()</code>	Removes the last returned element
<code>forEachRemaining()</code>	Processes remaining elements

12. ListIterator Interface

`ListIterator` allows traversal in both forward and backward directions.

Method	Description
<code>hasNext()</code>	Checks for a next element
<code>hasPrevious()</code>	Checks for a previous element
<code>next()</code>	Returns the next element
<code>previous()</code>	Returns the previous element
<code>nextIndex()</code>	Returns the next element's index
<code>previousIndex()</code>	Returns the previous element's index
<code>add()</code>	Adds an element
<code>set()</code>	Replaces an element
<code>remove()</code>	Removes an element



Algorithm / Logic Description

1. Import the required classes from `java.util`.
2. Create an `ArrayList` to demonstrate `Collection` and `List` methods.
3. Use `add()`, `addAll()`, `contains()`, `containsAll()`, `size()`, `remove()`, `removeAll()`, and `isEmpty()`.
4. Demonstrate `List`-specific methods such as `get()`, `set()`, `indexOf()`, `lastIndexOf()`, `subList()`, and `sort()`.
5. Create a `HashSet` to demonstrate `Set` operations and duplicate removal.

6. Create a `TreeSet` to demonstrate `SortedSet` and `NavigableSet` operations.
 7. Create a `Queue` using `LinkedList` and demonstrate queue operations.
 8. Create a `Deque` using `ArrayDeque` and demonstrate operations at both ends.
 9. Create a `HashMap` and demonstrate key-value operations.
 10. Create a `TreeMap` to demonstrate `SortedMap` and `NavigableMap` operations.
 11. Create an `Iterator` and traverse the collection.
 12. Create a `ListIterator` and traverse the list in forward and backward directions.
 13. Display the results of all operations.
-

Source Code

```
package tasks;

import java.util.*;

public class CollectionFrameworkDemo {

    public static void main(String[] args) {

        // =====
        // 1. COLLECTION AND LIST INTERFACE
        // =====

        System.out.println("==== COLLECTION AND LIST ====");

        List<String> list = new ArrayList<>();

        // add()
        list.add("Apple");
        list.add("Banana");
        list.add("Mango");
        list.add("Apple");

        System.out.println("List: " + list);

        // add(index, element)
        list.add(1, "Orange");
        System.out.println("After add(1, Orange): " + list);

        // addAll()
        List<String> extra = new ArrayList<>();
        extra.add("Grapes");
        extra.add("Pineapple");

        list.addAll(extra);
        System.out.println("After addAll(): " + list);

        // get()
        System.out.println("Element at index 2: " + list.get(2));
```

```

// set()
list.set(2, "Watermelon");
System.out.println("After set(): " + list);

// contains()
System.out.println("Contains Mango: "
    + list.contains("Mango"));

// containsAll()
System.out.println("Contains all extra elements: "
    + list.containsAll(extra));

// indexOf()
System.out.println("First index of Apple: "
    + list.indexOf("Apple"));

// lastIndexOf()
System.out.println("Last index of Apple: "
    + list.lastIndexOf("Apple"));

// size()
System.out.println("Size: " + list.size());

// subList()
System.out.println("SubList: "
    + list.subList(1, 4));

// sort()
list.sort(Comparator.naturalOrder());
System.out.println("Sorted List: " + list);

// remove(Object)
list.remove("Apple");
System.out.println("After remove(Apple): " + list);

// remove(index)
list.remove(0);
System.out.println("After remove(0): " + list);

// =====
// 2. SET INTERFACE
// =====

System.out.println("\n===== SET =====");

Set<Integer> set = new HashSet<>();

set.add(30);
set.add(10);
set.add(20);
set.add(10);

System.out.println("Set: " + set);

System.out.println("Contains 20: "
    + set.contains(20));

```

```

System.out.println("Set Size: " + set.size());

set.remove(30);
System.out.println("After remove(30): " + set);

System.out.println("Is Set Empty: "
    + set.isEmpty());

set.clear();
System.out.println("After clear(): " + set);

// =====
// 3. SORTED SET AND NAVIGABLE SET
// =====

System.out.println("\n===== SORTED SET / NAVIGABLE SET =====");

NavigableSet<Integer> numbers =
    new TreeSet<>();

numbers.add(10);
numbers.add(20);
numbers.add(30);
numbers.add(40);
numbers.add(50);

System.out.println("TreeSet: " + numbers);

System.out.println("First: " + numbers.first());
System.out.println("Last: " + numbers.last());

System.out.println("HeadSet(30): "
    + numbers.headSet(30));

System.out.println("TailSet(30): "
    + numbers.tailSet(30));

System.out.println("SubSet(20, 50): "
    + numbers.subSet(20, 50));

System.out.println("Lower(30): "
    + numbers.lower(30));

System.out.println("Floor(30): "
    + numbers.floor(30));

System.out.println("Ceiling(35): "
    + numbers.ceiling(35));

System.out.println("Higher(30): "
    + numbers.higher(30));

System.out.println("Descending Set: "
    + numbers.descendingSet());

```

```

System.out.println("Poll First: "
    + numbers.pollFirst());

System.out.println("Poll Last: "
    + numbers.pollLast());

System.out.println("After polling: " + numbers);

// =====
// 4. QUEUE
// =====

System.out.println("\n===== QUEUE =====");

Queue<String> queue = new LinkedList<>();

queue.add("A");
queue.add("B");
queue.offer("C");

System.out.println("Queue: " + queue);

System.out.println("Element: " + queue.element());
System.out.println("Peek: " + queue.peek());

System.out.println("Remove: " + queue.remove());
System.out.println("Poll: " + queue.poll());

System.out.println("Queue after operations: "
    + queue);

// =====
// 5. DEQUE
// =====

System.out.println("\n===== DEQUE =====");

Deque<String> deque = new ArrayDeque<>();

deque.addFirst("B");
deque.addLast("C");
deque.offerFirst("A");
deque.offerLast("D");

System.out.println("Deque: " + deque);

System.out.println("Peek First: "
    + deque.peekFirst());

System.out.println("Peek Last: "
    + deque.peekLast());

System.out.println("Remove First: "
    + deque.removeFirst());

```

```

System.out.println("Remove Last: "
    + deque.removeLast());

System.out.println("Poll First: "
    + deque.pollFirst());

System.out.println("Poll Last: "
    + deque.pollLast());

System.out.println("Deque after operations: "
    + deque);

// =====
// 6. MAP INTERFACE
// =====

System.out.println("\n===== MAP =====");

Map<Integer, String> map =
    new HashMap<>();

map.put(101, "Rahul");
map.put(102, "Priya");
map.put(103, "Akshaya");

System.out.println("Map: " + map);

System.out.println("Value for key 102: "
    + map.get(102));

System.out.println("Contains key 101: "
    + map.containsKey(101));

System.out.println("Contains value Rahul: "
    + map.containsValue("Rahul"));

System.out.println("Keys: " + map.keySet());

System.out.println("Values: " + map.values());

System.out.println("Entries: " + map.entrySet());

System.out.println("Map Size: " + map.size());

map.remove(103);
System.out.println("After remove(103): " + map);

System.out.println("Is Map Empty: "
    + map.isEmpty());

// =====
// 7. SORTED MAP / NAVIGABLE MAP
// =====

System.out.println("\n===== SORTED MAP / NAVIGABLE MAP =====");

```

```
NavigableMap<Integer, String> treeMap =
    new TreeMap<>();

treeMap.put(10, "Ten");
treeMap.put(20, "Twenty");
treeMap.put(30, "Thirty");
treeMap.put(40, "Forty");
treeMap.put(50, "Fifty");

System.out.println("TreeMap: " + treeMap);

System.out.println("First Key: "
    + treeMap.firstKey());

System.out.println("Last Key: "
    + treeMap.lastKey());

System.out.println("HeadMap(30): "
    + treeMap.headMap(30));

System.out.println("TailMap(30): "
    + treeMap.tailMap(30));

System.out.println("SubMap(20, 50): "
    + treeMap.subMap(20, 50));

System.out.println("Lower Key(30): "
    + treeMap.lowerKey(30));

System.out.println("Floor Key(30): "
    + treeMap.floorKey(30));

System.out.println("Ceiling Key(35): "
    + treeMap.ceilingKey(35));

System.out.println("Higher Key(30): "
    + treeMap.higherKey(30));

System.out.println("First Entry: "
    + treeMap.firstEntry());

System.out.println("Last Entry: "
    + treeMap.lastEntry());

System.out.println("Descending Map: "
    + treeMap.descendingMap());

System.out.println("Poll First Entry: "
    + treeMap.pollFirstEntry());

System.out.println("Poll Last Entry: "
    + treeMap.pollLastEntry());

System.out.println("After polling: "
    + treeMap);
```

```

// =====
// 8. ITERATOR
// =====

System.out.println("\n==== ITERATOR =====");

List<String> names = new ArrayList<>();

names.add("Rahul");
names.add("Priya");
names.add("Akshaya");
names.add("Kiran");

Iterator<String> iterator =
    names.iterator();

System.out.println("Using Iterator:");

while (iterator.hasNext()) {
    System.out.println(iterator.next());
}

// =====
// 9. LIST ITERATOR
// =====

System.out.println("\n==== LIST ITERATOR =====");

ListIterator<String> listIterator =
    names.listIterator();

System.out.println("Forward Traversal:");

while (listIterator.hasNext()) {
    System.out.println(
        listIterator.next()
    );
}

System.out.println("Backward Traversal:");

while (listIterator.hasPrevious()) {
    System.out.println(
        listIterator.previous()
    );
}

System.out.println("\nNext Index: "
    + listIterator.nextIndex());

System.out.println("Previous Index: "
    + listIterator.previousIndex());

System.out.println("\n==== PROGRAM COMPLETED =====");
}

```

```
}
```

Sample Output

```
===== COLLECTION AND LIST =====
List: [Apple, Banana, Mango, Apple]
After add(1, Orange): [Apple, Orange, Banana, Mango, Apple]
After addAll(): [Apple, Orange, Banana, Mango, Apple, Grapes, Pineapple]
Element at index 2: Banana
After set(): [Apple, Orange, Watermelon, Mango, Apple, Grapes, Pineapple]
Contains Mango: true
Contains all extra elements: true
First index of Apple: 0
Last index of Apple: 4
Size: 7
SubList: [Orange, Watermelon, Mango]
Sorted List: [Apple, Apple, Grapes, Mango, Orange, Pineapple, Watermelon]
After remove(Apple): [Apple, Grapes, Mango, Orange, Pineapple, Watermelon]
After remove(0): [Grapes, Mango, Orange, Pineapple, Watermelon]

===== SET =====
Set: [20, 10, 30]
Contains 20: true
Set Size: 3
After remove(30): [20, 10]
Is Set Empty: false
After clear(): []

===== SORTED SET / NAVIGABLE SET =====
TreeSet: [10, 20, 30, 40, 50]
First: 10
Last: 50
HeadSet(30): [10, 20]
TailSet(30): [30, 40, 50]
SubSet(20, 50): [20, 30, 40]
Lower(30): 20
Floor(30): 30
Ceiling(35): 40
Higher(30): 40
Descending Set: [50, 40, 30, 20, 10]
Poll First: 10
Poll Last: 50
After polling: [20, 30, 40]

===== QUEUE =====
Queue: [A, B, C]
Element: A
Peek: A
Remove: A
Poll: B
Queue after operations: [C]

===== DEQUE =====
```

```
Deque: [A, B, C, D]
Peek First: A
Peek Last: D
Remove First: A
Remove Last: D
Poll First: B
Poll Last: C
Deque after operations: []
```

===== MAP =====

```
Map: {101=Rahul, 102=Priya, 103=Akshaya}
Value for key 102: Priya
Contains key 101: true
Contains value Rahul: true
Keys: [101, 102, 103]
Values: [Rahul, Priya, Akshaya]
Entries: [101=Rahul, 102=Priya, 103=Akshaya]
Map Size: 3
After remove(103): {101=Rahul, 102=Priya}
Is Map Empty: false
```

===== SORTED MAP / NAVIGABLE MAP =====

```
TreeMap: {10=Ten, 20=Twenty, 30=Thirty, 40=Forty, 50=Fifty}
First Key: 10
Last Key: 50
HeadMap(30): {10=Ten, 20=Twenty}
TailMap(30): {30=Thirty, 40=Forty, 50=Fifty}
SubMap(20, 50): {20=Twenty, 30=Thirty, 40=Forty}
Lower Key(30): 20
Floor Key(30): 30
Ceiling Key(35): 40
Higher Key(30): 40
First Entry: 10=Ten
Last Entry: 50=Fifty
Descending Map: {50=Fifty, 40=Forty, 30=Thirty, 20=Twenty, 10=Ten}
Poll First Entry: 10=Ten
Poll Last Entry: 50=Fifty
After polling: {20=Twenty, 30=Thirty, 40=Forty}
```

===== ITERATOR =====

```
Using Iterator:
Rahul
Priya
Akshaya
Kiran
```

===== LIST ITERATOR =====

```
Forward Traversal:
Rahul
Priya
Akshaya
Kiran
Backward Traversal:
Kiran
Akshaya
Priya
Rahul
```

Next Index: 0
Previous Index: -1

===== PROGRAM COMPLETED =====

Note about `HashSet` **and** `HashMap` **output**

`HashSet` and `HashMap` **do not guarantee iteration order**, so their displayed element order may differ from the sample output. The important point is that the demonstrated methods produce the correct results.
